

Unit 6: Swing

Topics Covered: Basic swing components : Text Fields, Buttons, Toggle Buttons, Checkboxes, and Radio Buttons. Event Handling: various event handling mechanisms, Delegation Event Model, Events, Event Sources, Event Listeners, various classes related to event sources and event listeners.

1. Introduction to Swing

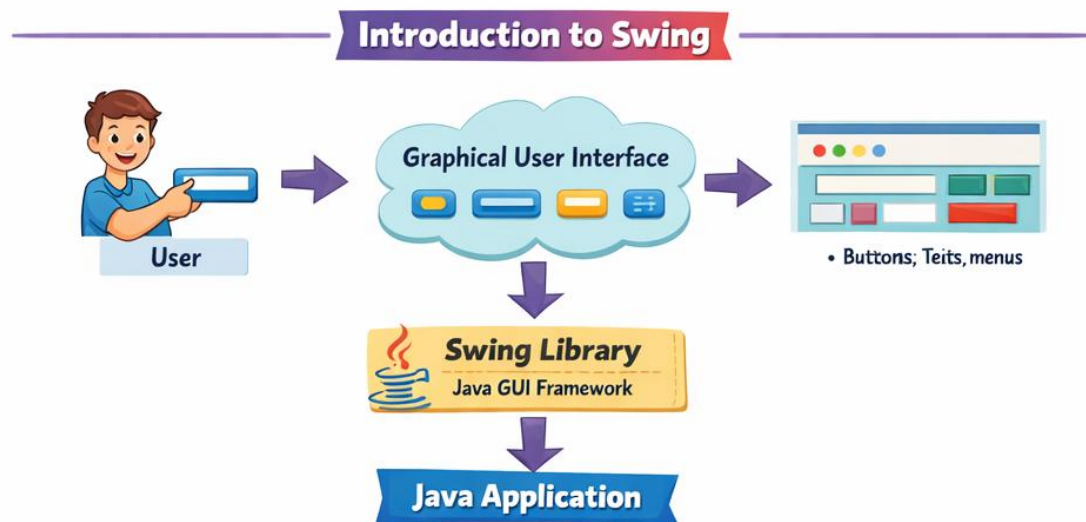


Fig. 1: Introduction to Swing

Modern applications allow users to interact visually with software through windows, buttons, text fields, and menus. This type of interaction is known as a **Graphical User Interface (GUI)**.

Java provides several libraries for creating GUI applications. One of the most commonly used libraries is Swing which can be seen in Fig. 1.

Swing components belong to the package:

`javax.swing`

Therefore, most Swing programs begin with:

```
import javax.swing.*;
```

Key Idea:

Swing is a Java library used to create graphical user interfaces (GUI) for desktop applications.

Why Swing is Useful

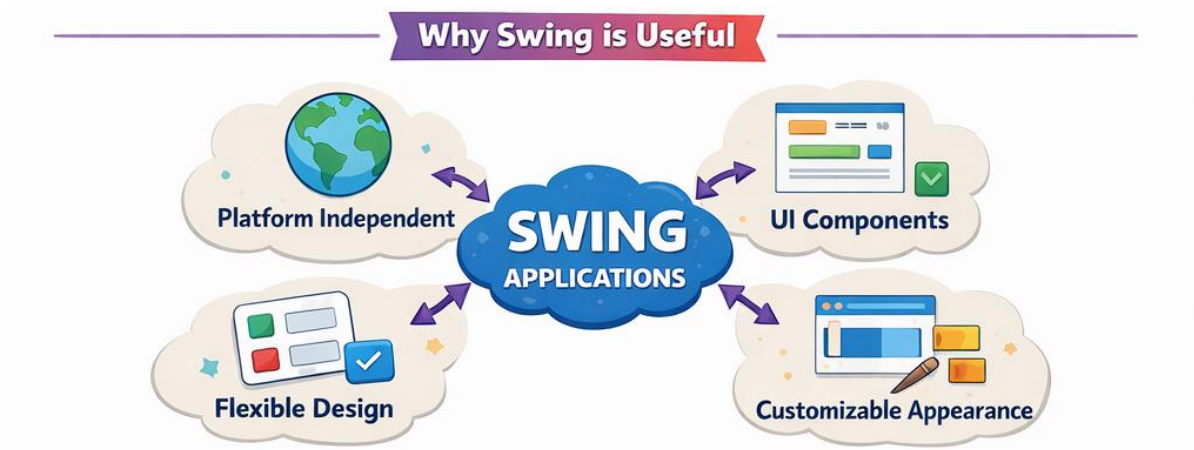


Fig. 2: Advantages of Swing

Swing provides several advantages for building GUI applications as seen in Fig. 2:

- Platform-independent interface
- Large collection of ready-to-use components
- Flexible design of user interfaces
- Support for customizable appearance

Because of these features, Swing is widely used to develop **desktop applications**.

Applications of Swing

Swing is mainly used to build **desktop software** as seen in Fig. 3. Some common examples include:

- Calculator applications
- Text editors
- Student management systems
- Banking software
- Database tools
- IDEs such as NetBeans



Fig. 3: Applications of Swing

Basic Swing Components

Swing provides many graphical components that can be used to design user interfaces. Some of the most commonly used components are:

- JFrame
- JTextField
- JButton
- JToggleButton
- JCheckBox
- JRadioButton

These components are placed inside a window to create the graphical interface of the program.

JFrame (Main Window)

JFrame represents the **main window of a Swing application**.

All other components are added inside this window.

Example:

```
import javax.swing.*;

public class FrameExample {

    public static void main(String[] args) {

        // Create a frame with a title

        JFrame frame = new JFrame("Swing Example");

        // Set size of window

        frame.setSize(400,300);

        // Make window visible

        frame.setVisible(true);

    }

}
```

Explanation:

- JFrame creates the window.
- setSize() defines the width and height.
- setVisible(true) displays the window.

Output:

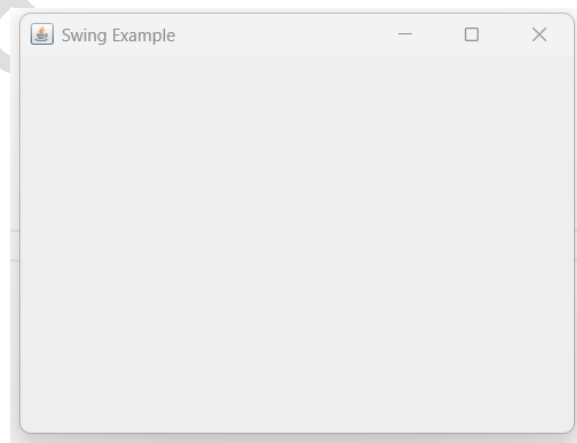


Fig. 4 JFram Example

JTextField

A **JTextField** is used to accept **single-line input from the user**.

It is commonly used for entering values such as:

- names
- numbers
- search queries

Example:

```
JTextField text = new JTextField();  
text.setBounds(50,50,200,30);
```

Explanation: `setBounds(x,y,width,height)` specifies the **position and size of the component**.

JButton

A **JButton** represents a clickable button in a graphical interface.

Buttons are typically used to perform actions such as **submit, calculate, or clear data**.

Example:

```
JButton b = new JButton("Click");  
b.setBounds(50,100,100,30);
```

However, simply displaying a button does not perform any action.

To respond to user interaction, we must use **event handling**.

Toggle Buttons, Checkboxes, and Radio Buttons

Swing provides additional components for selecting options.

JToggleButton

A **JToggleButton** switches between two states such as **ON and OFF**.

Example:

```
JToggleButton toggle = new JToggleButton("Power");
```

JCheckBox

A **JCheckBox** allows the user to select **multiple options simultaneously**.

Example:

```
JCheckBox c1 = new JCheckBox("Java");  
JCheckBox c2 = new JCheckBox("Python");
```

Users can select **one or more options**.

JRadioButton

A **JRadioButton** allows the user to select **only one option from a group**.

Example:

```
JRadioButton r1 = new JRadioButton("Male");  
JRadioButton r2 = new JRadioButton("Female");  
ButtonGroup bg = new ButtonGroup();  
bg.add(r1);  
bg.add(r2);
```

The ButtonGroup ensures that **only one radio button can be selected at a time**.

10. Delegation Event Model

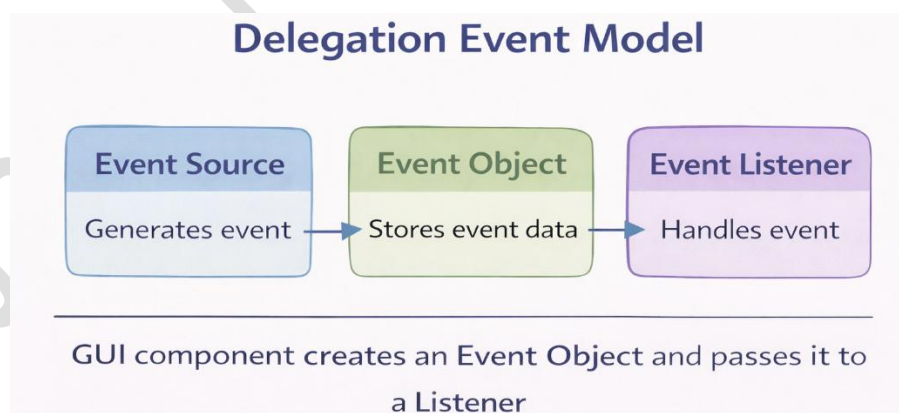


Fig. 5 Delegation event Model

In a GUI application, the user interacts with components such as **buttons, text fields, or check boxes**. When the user performs an action (like clicking a button), an **event** is generated.



Java handles this interaction using the **Delegation Event Model**. This model separates **event generation** from **event handling**. In this model, These components work together to detect and handle user actions.

1. **Event Source**
2. **Event Object (instance of event class)**
3. **Event Listener**

1. Event Source

The **Event Source** is the component that **generates the event** when a user interacts with it.

Examples of event sources include:

- JButton
- JTextField
- JCheckBox
- JRadioButton

Example:

```
JButton button = new JButton("Add");
```

If the user clicks this button, the **button becomes the event source**.

2. Event Class and Event Object

In the **Delegation Event Model**, when a user interacts with a GUI component (for example clicking a button), Java generates an event. This event is represented using **event classes and event objects**.

2.1 Event Class

An **Event Class** defines the **type of event that can occur in a GUI application**.

It acts as a blueprint describing the event.

Event classes are located in the package:

```
java.awt.event
```

Examples of event classes include:

Table 1- Event Class examples

Event Class	Generated When
ActionEvent	Button is clicked
MouseEvent	Mouse actions occur
KeyEvent	Keyboard key is pressed
ItemEvent	Checkbox or radio button state changes
WindowEvent	Window operations occur

2.2 Event Object

An **Event Object** is an **instance of an event class that is created when the event actually occurs.**

It contains information about the event and is passed to the event listener.

Example:

```
public void actionPerformed(ActionEvent e)
```

Here:

- **ActionEvent** → Event Class
- **e** → Event Object

The object e stores details such as **which component generated the event.**

2.3 Difference Between Event Class and Event Object

Table 2- Difference between event class and event object

Event Class	Event Object
Defines the type of event	Represents a specific occurrence of the event
Acts as a blueprint	Created from the event class

Event Class	Event Object
Example: <code>ActionEvent</code>	Example: <code>ActionEvent e</code>

3. Event Listener

An **Event Listener** is responsible for **handling the event**.

It waits for the event to occur and then executes the required code.

Example:

```
button.addActionListener(this);
```

Here the program is saying:

“Whenever this button is clicked, run the event handling code.”

The listener method is:

```
public void actionPerformed(ActionEvent e)
```

Simple Flow of Event Handling



Code Example:

```
JButton button = new JButton("Click");
```

```
button.addActionListener(this);

public void actionPerformed(ActionEvent e){

    System.out.println("Button clicked");

}
```

Explanation:

- JButton → Event Source
- ActionEvent → Event Class
- e → Event Object
- ActionListener → Event Listener

Key Idea:

Event handling in Swing allows programs to respond to user actions.

- **Event Source** → component that generates the event
- **Event Object** → contains information about the event
- **Event Listener** → handles the event and performs the required action

This mechanism enables Swing applications to **interact with users and respond dynamically to their actions**

Developing a Calculator Application using Swing

To understand Swing components and event handling together, we can create a **simple calculator application**.

Code:

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class CalculatorSwing extends JFrame implements ActionListener {
```



```
JTextField display;      // calculator display

String operator = "";    // stores operator

double num1 = 0, num2 = 0; // numbers for calculation


CalculatorSwing() {

    setTitle("Calculator");

    setSize(300,350);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    setLayout(new BorderLayout());


    display = new JTextField();      // display field

    display.setFont(new Font("Arial",Font.BOLD,20));

    display.setHorizontalAlignment(JTextField.RIGHT);

    add(display, BorderLayout.NORTH);


    JPanel panel = new JPanel();

    panel.setLayout(new GridLayout(5,4,5,5));    // button grid


    String buttons[] = {

        "7","8","9","/",

        "4","5","6","*",

        "1","2","3","-",

        "0","C","=","+"

    };


    for(String text : buttons)

    {
```



```
Jbutton btn = new Jbutton(text);

btn.setFont(new Font("Arial",Font.BOLD,16));

btn.addActionListener(this);           // button click event

panel.add(btn);

}

add(panel, BorderLayout.CENTER);

setVisible(true);

}

public void actionPerformed(ActionEvent e) {

    String command = e.getActionCommand();

    if(command.matches("[0-9]")) {      // number pressed

        display.setText(display.getText()+command);

    }

    else if(command.equals("C")) {      // clear display

        display.setText("");

        num1 = num2 = 0;

        operator = "";

    }

    else if(command.matches("[+\\-*/]")) { // operator pressed

        num1 = Double.parseDouble(display.getText());

        operator = command;

        display.setText("");

    }

    else if(command.equals("=")) {      // calculate result
```

```
num2 = Double.parseDouble(display.getText());

switch(operator) {

    case "+": display.setText(""+(num1+num2)); break;
    case "-": display.setText(""+(num1-num2)); break;
    case "*": display.setText(""+(num1*num2)); break;
    case "/": display.setText(""+(num1/num2)); break;

}

}

}

public static void main(String[] args) {

    new CalculatorSwing();

}

}
```

Output:

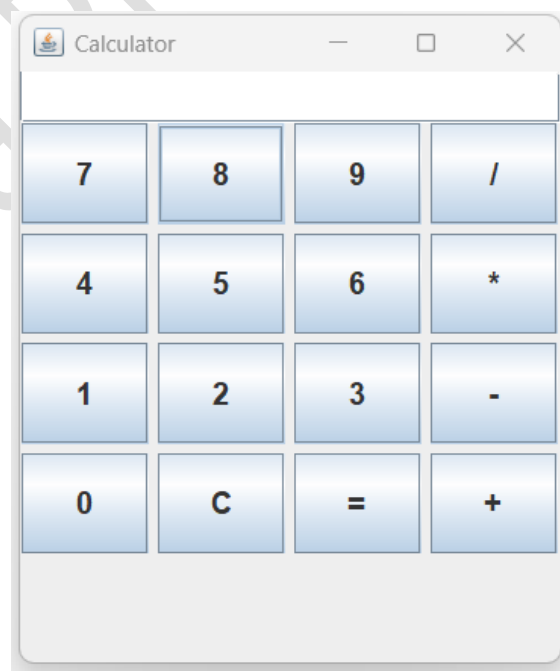


Fig. 6 : Output of program

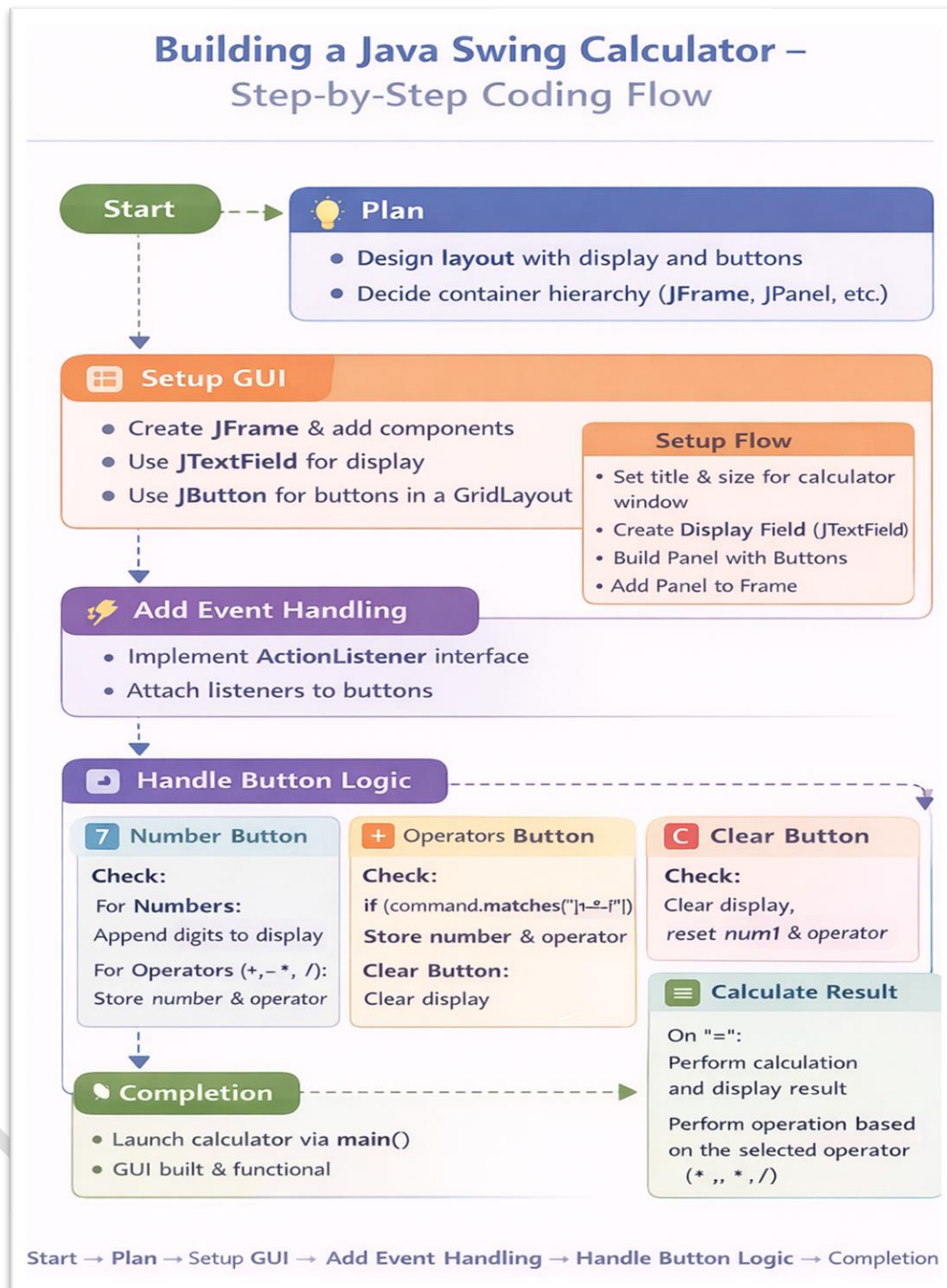


Fig. 7: Step by Step Code flow